Estructuras de Datos

2026
Primer Cuatrimestre

REPASO

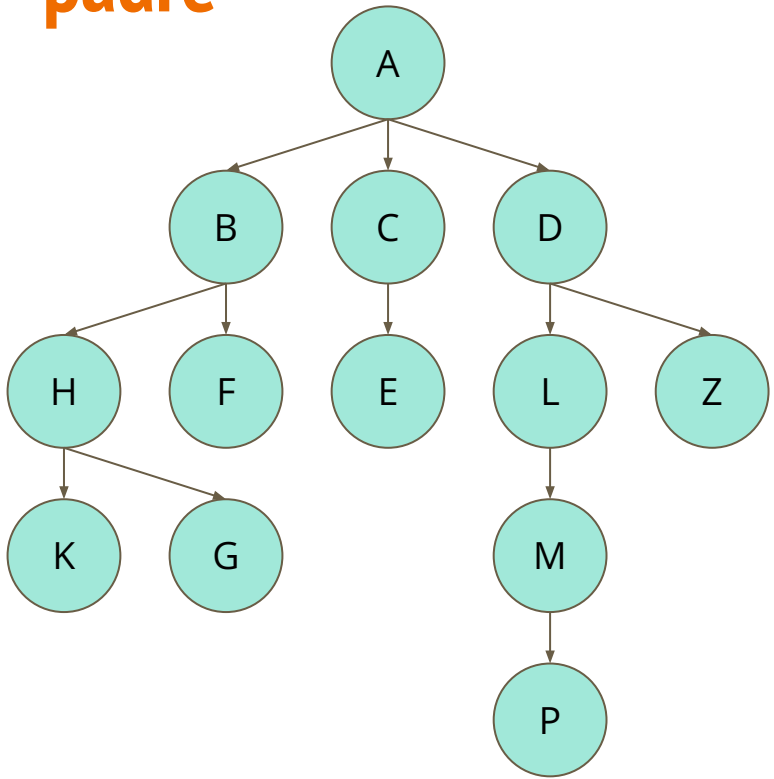
TDA ÁRBOL - Interface

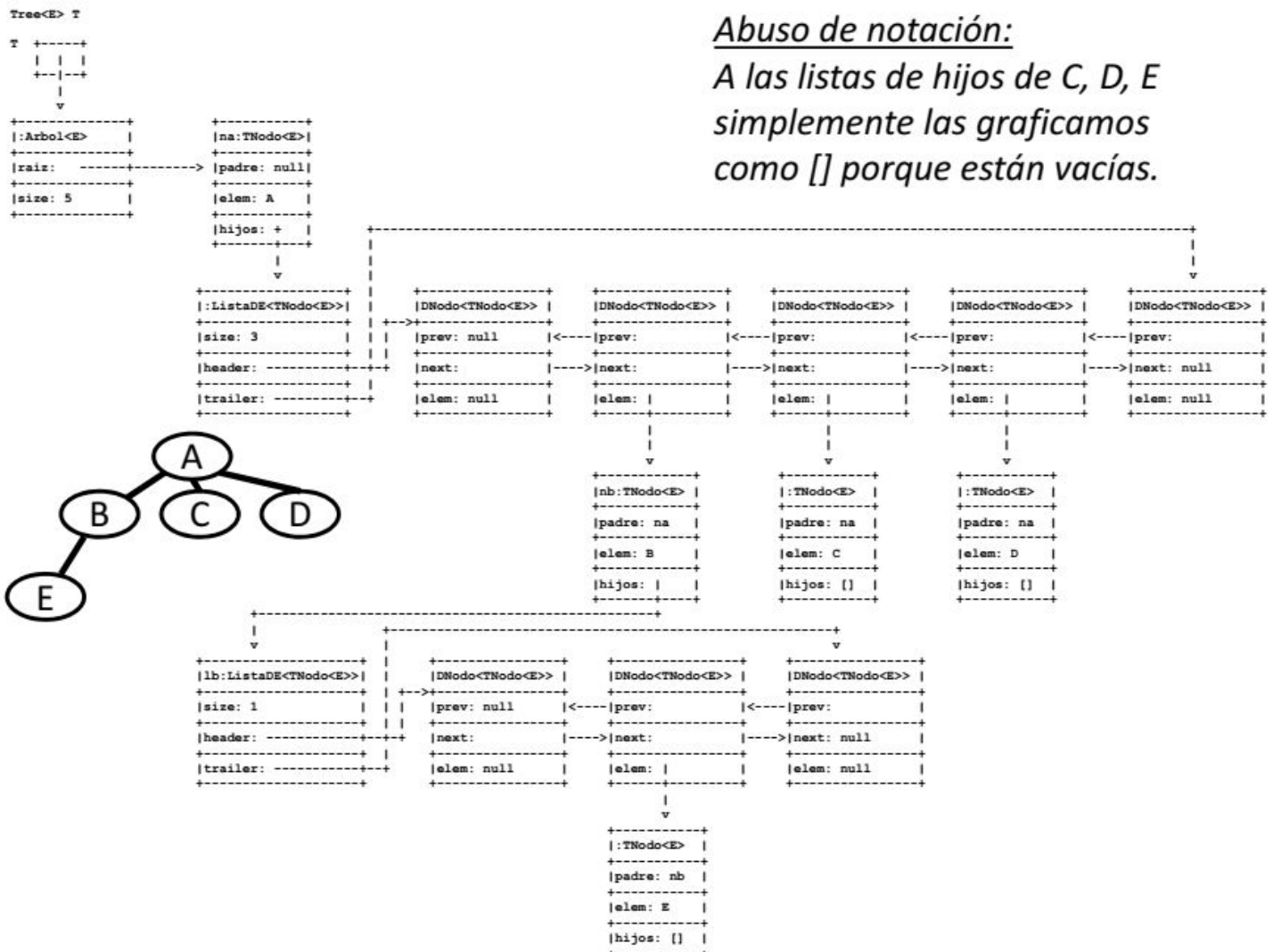
```
public interface Tree<E> extends Iterable<E>
{
public int size();
public boolean isEmpty();
public Iterator<E> iterator();
public Iterable<Position<E>> positions();
public E replace(Position<E> v, E e) throws InvalidPositionException;
public Position<E> root() throws EmptyTreeException;
public Position<E> parent(Position<E> v) throws InvalidPositionException, BoundaryViolationException;
public Iterable<Position<E>> children(Position<E> v) throws InvalidPositionException;
public boolean isInternal(Position<E> v) throws InvalidPositionException;
public boolean isExternal(Position<E> v) throws InvalidPositionException;
public boolean isRoot(Position<E> v) throws InvalidPositionException;
public void createRoot(E e) throws InvalidOperationException;
public Position<E> addFirstChild(Position<E> p, E e) throwsInvalidPositionException;
public Position<E> addLastChild(Position<E> p, E e) throws InvalidPositionException;
public Position<E> addBefore(Position<E> p, Position<E> rb, E e) throws InvalidPositionException;
public Position<E> addAfter (Position<E> p, Position<E> lb, E e) throws InvalidPositionException;
public void removeExternalNode (Position<E> p) throws InvalidPositionException;
public void removeInternalNode (Position<E> p) throws InvalidPositionException;
public void removeNode (Position<E> p) throws InvalidPositionException;
}
```

Representación de lista de hijos + padre

Arbol<E> implements Tree<E>
raiz: TNodo<E> size: integer

TNodo<E> implements Position<E>
elem: E padre: TNodo<E> hijos: PositionList<TNodo<E>>





<u>Abuso de notación:</u>

A las listas de hijos de C, D, E simplemente las graficamos como [] porque están vacías.

Recordar

- Plantear estrategia primero

Fuera de la estructura

- Operaciones del TDA solamente
- Manejo de iteradores
  - Elementos
  - Posiciones
- Puedo llegar a hacer mis propios recorridos

Dentro de la estructura

- Ver si se tiene acceso a las operaciones de TDA y a cuáles
  - Potencial uso de iteradores (muy ocasional)
  - Más importante saber realizar recorridos
- Recordar atributos y operaciones de otros TDAs relacionados
  - TNodo - Position

PROBLEMAS

Problema 1

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea **public int sizeSubarbol(Position<E> p)**. Este método deberá retornar el tamaño del subárbol con raíz p del árbol receptor del mensaje. Si utiliza otros métodos del TDA Arbol deberá implementarlos. Se debe lanzar **InvalidPositionException** cuando corresponda.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Estrategia

- ¿Dentro o fuera de la estructura? ¿Puedo usar o no el TDA?
- Chequear:
  - ¿Qué pasa si el árbol es vacío o la posición es inválida? → Check position
    - p==null, this.isEmpty, try/catch con casteo a TNodo
- ¿Cómo puedo contar el subárbol?
  - No recursivo → Necesito un TDA/estructura auxiliar
    - Itero hasta recorrer todo el subárbol
    - Cuento la raíz del subárbol y pongo a los hijos de p en una cola
    - Desencolo y sumo uno
      - Si tiene hijos los pongo en la cola
    - Sigo iterando hasta que me quede la cola vacía
  - Recursivo
    - Caso base:
      - El nodo es hoja y el resultado a devolver es 1
    - Caso recurrente
      - El nodo es interno, el resultado es 1 + el tamaño de todos los subárboles con raiz en los hijos

Problema 2

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea **public Map<Position<E>, Integer> mapSizeSubarboles( )**. Este método deberá retornar un mapeo entre posiciones del árbol y el tamaño del subárbol que tiene a esa posición como raíz. Si utiliza otros métodos del TDA Arbol deberá implementarlos. Se debe lanzar **InvalidPositionException** cuando corresponda.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Estrategia

- ¿Dentro o fuera de la estructura? ¿Puedo usar o no el TDA?
- Chequear:
  - ¿Que pasá si el árbol es vacío? → devuelvo mapeo vacío
- ¿Cómo puedo contar el subárbol?
  - No eficiente
    - Recorro todas las posiciones
    - Se la paso al método anterior
    - Y agrego en el mapeo la entrada
    - $O(n^2)$
  - Eficiente en el mismo recorrido cuento y agrego al mapeo
    - Caso base:
      - El nodo es hoja y agrego al mapeo (nodoActual, 1)
    - Caso recurrente
      - El nodo es interno, y agrego al mapeo (nodoActual, 1 + el tamaño de todos los subárboles con raiz en los hijos)

Problema 3

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase. Esta clase utiliza la implementación de lista de hijos y enlace al padre. Agregue un método a esta clase cuya signatura sea public **int podarSubarbol(Position<E> p)**. Este método deberá eliminar el subarbol con raíz p del árbol receptor del mensaje. Retorna el tamaño del subárbol eliminado. Si utiliza otros métodos del TDA Arbol deberá implementarlos. Se debe lanzar **InvalidPositionException** cuando corresponda.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Estrategia

- ¿Dentro o fuera de la estructura? ¿Puedo usar o no el TDA?
- Chequear:
  - ¿Que pasá si el árbol es vacío o la posición es inválida? → Check position
    - p==null, this.isEmpty, try/catch con casteo a TNodo
- ¿Cómo puedo podar el subárbol?
  - Tengo la posición del Tnodo → Tengo que eliminar el elemento de la posición de la lista (DNodo) que contiene a TNodo
    - Tengo que acceder al padre
    - Buscar al nodo hijo
    - Sacarlo de la lista
    - Descontar la cantidad de nodos del subárbol removido y devolver esa cantidad
  - ¿Qué pasa con la raíz? → No tiene padre —> trabajar como caso particular

Problema 4

Suponiendo que cuenta con una clase *Arbol<E>* que implementa la interfaz *Tree<E>* vista en clase y trabajando en función de las operaciones del TDA Árbol, resuelva el siguiente problema; dado un árbol *T* y dos rótulos *e y f*, se le debe cambiar el rótulo a todos los nodos del árbol que tengan el rótulo *e* por *f*.

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Estrategia

- ¿Dentro o fuera de la estructura? ¿Puedo usar o no el TDA?
- Chequear:
  - ¿Que pasá si el árbol es vacío? → Nada
- ¿Cómo puedo intercambiar elementos del subárbol?
  - ¿Requiero de algún orden en particular? ¿Tengo que modificar la estructura? → Uso iterador de posiciones - Recorrido exhaustivo
    - Si el elemento es el buscado
    - Usar replace (TDA Arbol)
  - Cuidado con el equals y chequeo de null

Problema 5

Utilizando el TDA Árbol, resuelva el siguiente problema; dado un árbol A, un rótulo *e* y un nivel *x*, se le debe insertar un nuevo primer hijo con rótulo *e* a cada nodo en el nivel *x*. El método a implementar debe tener la siguiente forma:

public static void insertarMasivo(E e, int x, Tree<E> A)

Calcular el orden del tiempo de ejecución de su solución justificando adecuadamente.

Estrategia

- ¿Dentro o fuera de la estructura? ¿Puedo usar o no el TDA?
- Chequear:
  - ¿Que pasa si el árbol es vacío? → Nada
  - ¿Qué pasa si el nivel dado no está en el árbol → Nada
- ¿Cómo puedo hacer esa inserción?
  - ¿Necesito un recorrido por niveles? → No necesariamente
  - Recorro el árbol llevando cuenta del nivel
    - Caso base:
      - Si el nodo tiene el nivel buscado, inserto a e como primer hijo.
      - Termina el recorrido
    - Caso recursivo
      - Si el nodo no tiene el nivel buscado, avanzo por el nivel de los hijos para ver si puedo insertar ahí.

THE END.